



Rapport Projet : minishell

Mathis Magron

Année universitaire 2024/2025

Table des matières

1	Introduction	2
2	Architecture de l'application	2
3	Gestion des processus	2
4	Manipulation de signaux et terminaison de processus	3
5	Masque de signaux - Signaux par défaut	4
6	Fichiers et Redirection	4
6.1	Redirection des entrées/sorties d'un processus	4
6.2	Manipulation des répertoires	5
7	Tubes	6
8	Bonus : Contrôle des E/S - Mémoire	6
9	Conclusion personnelle	6

1 Introduction

Ce projet vise à permettre de mettre en pratique les notions de base, autour de la gestion des processus, des signaux et des E/S, dans un contexte suffisamment réaliste. Il s'agit plus précisément de développer un interpréteur de commandes simplifié, offrant les fonctionnalités de base des shells Unix, comme le bash.

2 Architecture de l'application

Le code de l'application est entièrement contenu dans le fichier `"minishell.c"`. A l'intérieur on y retrouve différentes méthodes :

1. `"void redirections(char* entree, char* sortie)"` : elle permet d'associer l'entrée standard ou la sortie standard d'une commande à un fichier.
2. `"void traitement(int sig)"` : traite le signal `SIGCHLD` qui gère la terminaison de processus fils et affiche des informations différentes selon que le fils s'est : terminé (`WIFEXITED`) , suspendu (`WIFSTOPPED`), repris (`WIFCONTINUED`).
3. `"void arborescence(char* *chemin)"` : permet de changer le répertoire courant vers un répertoire indiqué en argument, en utilisant la primitive `chdir`. Lorsque l'argument de la fonction vaut `NULL`, le répertoire courant devient le répertoire racine de l'utilisateur, identifié par la variable d'environnement `HOME`.
4. `"void affiche_contenu(char* *repertoire)"` : affiche le contenu d'un répertoire passé en paramètre. Si la commande `dir` est exécutée sans argument, elle affiche le contenu du répertoire courant.
5. `"main"` : fonction principale du shell. Elle appelle `readcmd()` pour parser la ligne. Puis elle est divisée en 2 grandes parties : commandes avec pipelines et commandes sans pipeslines afin de traiter de la bonne façon.

3 Gestion des processus

L'utilisation du caractère `'&'` permet de mettre un processus en arrière plan

Sur la Figure 1, l'on observe que le processus de la commande `sleep 10 &` s'exécute bien en arrière plan car il est possible d'effectuer d'autres commandes en même temps que celui ci.

```

> sleep 10 &
commande : sleep 10
> ps fj
commande : ps fj

```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
1299063	1299156	1299156	1299156	pts/3	1323128	Ss	57826	0:00	/usr/bin/bash --in
1299156	1323128	1323128	1299156	pts/3	1323128	S+	57826	0:00	_ ./minishell
1323128	1326937	1326937	1299156	pts/3	1323128	S	57826	0:00	_ sleep 10
1323128	1326999	1323128	1299156	pts/3	1323128	R+	57826	0:00	_ ps fj
1298839	1298845	1298845	1298845	pts/2	1298845	Ss+	57826	0:00	bash

FIGURE 1 – Exemple d’test d’une commande en arrière plan

4 Manipulation de signaux et terminaison de processus

La seconde étape importante est d’ajouter la définition d’un gestionnaire de signal, l’utilisation de `waitpid` pour gérer la terminaison des processus fils, et l’envoi de signaux pour suspendre et reprendre les processus en arrière-plan.

La Figure 2 illustre ce que le minishell peut renvoyer. Veuillez vous référer à l’implémentation de la fonction `traitement`

```

> sleep 100 &
commande : sleep 100
> ps fjn
commande : ps fjn

```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
1330351	1330412	1330412	1330412	pts/2	1341688	Ss	57826	0:00	/usr/bin/bash --ini
1330412	1341688	1341688	1330412	pts/2	1341688	S+	57826	0:00	_ ./minishell
1341688	1344308	1344308	1330412	pts/2	1341688	T	57826	0:00	_ sleep 100
1341688	1345401	1345401	1330412	pts/2	1341688	S	57826	0:00	_ sleep 100
1341688	1345437	1341688	1330412	pts/2	1341688	R+	57826	0:00	_ ps fjn
1330168	1342024	1342024	1342024	pts/1	1342024	Ss+	57826	0:00	bash
1330168	1330178	1330178	1330178	pts/0	1330178	Ss+	57826	0:00	bash

```

Reception du signal 17
Le processus de pid : 1345437 vient de se terminer
> kill -SIGSTOP 1345401
commande : kill -SIGSTOP 1345401

Reception du signal 17
Le processus de pid : 1345401 vient d'être suspendu avec le signal SIGSTOP

```

FIGURE 2 – Exemple de message de retour lors de l’utilisation de `kill`

Ici grâce à la méthode `void traitement(int sig)` le minishell nous dit que le processus `sleep 100`, en arrière plan, à bien était stoppé par l’envoi du signal `SIGSTOP`, il entre ainsi en état "T".

5 Masque de signaux - Signaux par défaut

Pour cette partie, nous avons implémenter le comportement des processus fils en utilisant l'appui au clavier de ctrl-C et ctrl-Z. Le traitement normal est d'arrêter ou suspendre le processus en avant-plan. Pour ce faire on utilise la primitive `setpgrp()` qui associe une nouveau groupe au processus appelant ainsi que la primitive `int sigprocmask(int how, const sigset_t *set, sigset_t *old_set)` pour masquer les signaux SIGINT et SIGTSTP.

Voici un exemple de test du ctrl-Z :

```
> sleep 100 &
commande : sleep 100
> sleep 100
commande : sleep 100
^Z
Reception du signal 17
Le processus de pid : 1351575 vient d'être suspendu avec le signal SIGSTOP
>
```

FIGURE 3 – Exemple d'utilisation du ctrl-Z

Le processus en avant plan est bien arrêté tandis que celui en arrière plan continue.

6 Fichiers et Redirection

6.1 Redirection des entrées/sorties d'un processus

Par la suite, nous avons mis en place dans notre programme, une redirection qui permet d'associer l'entrée standard ou la sortie standard d'une commande à un fichier.

C'est la méthode `void redirections(char* entree, char* sortie)` qui l'implémente.

Voici un exemple d'utilisation possible : `cat <titi.txt> toto.txt` associe `titi.txt` à l'entrée standard de `cat`, et `toto.txt` à la sortie standard de `cat`

```
> cat <titi.txt> toto.txt
commande : cat

Reception du signal 17
Le processus de pid : 1355858 vient de se terminer
erreur lecture commande
: Interrupted system call
```

FIGURE 4 – Enter Caption

Ici, nous avons une erreur qui n'a pas réussi à être corrigée mais le cat a bien lieu et le contenu de `titi.txt` est bien copié dans celui de `toto.txt`.

6.2 Manipulation des répertoires

Maintenant nous voulons ajouter une fonction qui permet de changer le répertoire courant vers un répertoire indiqué en argument, en utilisant la primitive `chdir`. C'est la fonction `void arborescence(char* *chemin)` du fichier `minishell.c`.

Ainsi qu'une fonction qui affiche le contenu d'un répertoire passé en paramètre. Si la commande `dir` est exécutée sans argument, elle affiche le contenu du répertoire courant. C'est la fonction `void affiche_contenu(char* *repertoire)` du fichier `minishell.c`.

Voici deux exemples d'utilisations :

```
> ls
commande : ls
Makefile  minishell.c  readcmd.c  readcmd.o  test_readcmd.c  toto.txt
minishell minishell.o readcmd.h  repertoire titi.txt

Reception du signal 17
Le processus de pid : 1359886 vient de se terminer
> cd repertoire
> ls
commande : ls
'salut je suis la'

Reception du signal 17
Le processus de pid : 1359947 vient de se terminer
> █
```

(a) Exemple de test de la commande `cd`

```
> dir
.
..
.vscode
titi.txt
toto.txt
readcmd.o
minishell.o
Makefile
readcmd.h
test_readcmd.c
minishell.c
repertoire
minishell
readcmd.c
> █
```

(b) Exemple de test de la commande `dir`

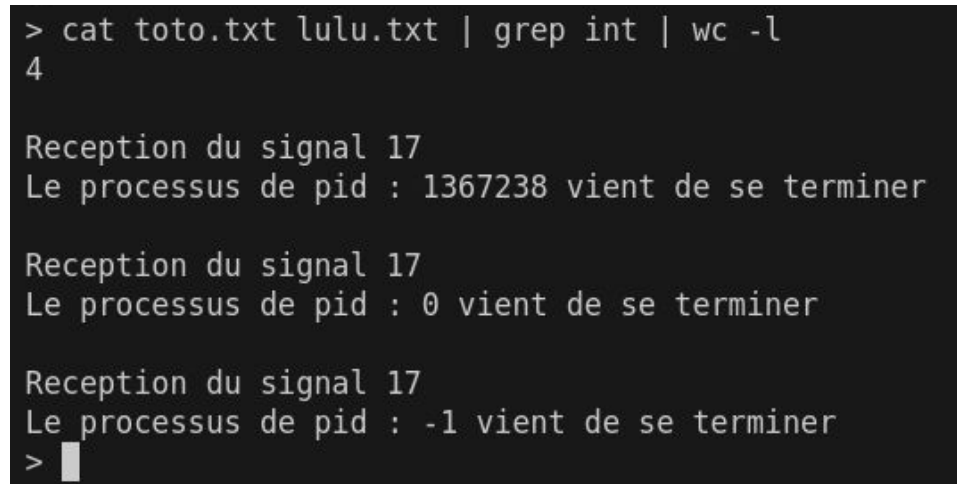
7 Tubes

Dans cette dernière section concernant notre minishell, l'objectif est de pouvoir composer des commandes en les reliant par un tube et d'étendre cette fonctionnalité en offrant la possibilité d'enchaîner une séquence de filtres liés par des tubes, de sorte à obtenir un traitement en pipeline.

Testons son implémentation avec la commande suivante :

```
cat toto.txt lulu.txt | grep int | wc -l.
```

Cette commande doit retourner le nombre total de lignes contenant le mot "int" dans les fichiers toto.txt et lulu.txt. Dans le test suivant, toto.txt contient 3 lignes avec le mot int, et titi.txt une seule. Le résultat attendu est donc quatre.



```
> cat toto.txt lulu.txt | grep int | wc -l
4

Reception du signal 17
Le processus de pid : 1367238 vient de se terminer

Reception du signal 17
Le processus de pid : 0 vient de se terminer

Reception du signal 17
Le processus de pid : -1 vient de se terminer
> █
```

FIGURE 6 – Test des pipelines

Nous obtenons bien le bon résultat.

8 Bonus : Contrôle des E/S - Mémoire

Le TP6 a été réalisé, le code est disponible dans l'archive.

9 Conclusion personnelle

Le projet de développement d'un minishell m'a permis d'appliquer des concepts clés des systèmes d'exploitation, tels que la gestion des processus, des signaux, et des entrées/sorties. En implémentant un shell Unix simplifié, j'ai exploré des fonctionnalités essentielles comme la redirection des flux, la manipulation des répertoires, et l'utilisation de tubes pour les pipelines.

NB : tout les fichiers utilisés pour les tests sont disponibles dans l'archive.